Aurore et le trésor d'Aurore

Table des matières

Analyse préliminaire 4
1.1 Introduction 4
1.2 Objectifs 4
Planification initiale 5

Analyse / Conception 5
Concept 5
Maquettes : 6
Diagramme de Classe : 8
Choix de code : 8
Stratégie de test 8
Risques techniques 8
Planification 9
Dossier de conception 9

Réalisation 10
Dossier de réalisation 10
Description des tests effectués plus résultats des tests d'acceptations 10
Mouvement du joueur (flèche du haut) : 10
Mouvement du joueur (flèche du bas) : 11
Mouvement du joueur (flèche de gauche) : 11
Mouvement du joueur (flèche de droite) : 11
Collision entre le joueur et les bords de l'écran : 12
Collision entre le joueur et les objets : 12
Ennemi qui suit le joueur lorsqu'il est proche par le haut : 13
Ennemi qui suit le joueur par le bas : 13
Ennemi qui suit le joueur par la gauche : 14
Ennemi qui suit le joueur lorsqu'il est proche par la droite : 14
Collision entre l'ennemi et le joueur : 15
Ennemi qui se trouve loin du joueur : 16
Changement de map depuis le haut de l'écran : 16
Changement de map depuis le bas de l'écran : 17
Changement de map depuis la gauche de l'écran : 17
Changement de map depuis la droite de l'écran : 18
Map non connecté par le haut : 18
Map non connecté par le bas : 19
Map non connecté par la gauche : 19

Map non connecté par la droite : 20
Affichage de l'inventaire :..... 20
Inventaire vide : 21
Sélection des items dans l'inventaire :..... 21
Utilisation de l'item puis disparition de celui-ci :..... 22
Ramassage des items plus stockage dans l'inventaire :..... 22
Erreurs restantes..... 25
Liste des documents fournis..... 25

Conclusions 25

Annexes..... 26
Résumé du rapport du TPI / version succincte de la documentation..... 26
Sources – Bibliographie 26
Journal de travail..... 26
Manuel d'Installation..... 27
Manuel d'Utilisation 27
Archives du projet..... 27

Analyse préliminaire

1.1 Introduction

Ce projet consiste à créer un jeu vidéo en 2D dans lequel nous incarnerons une aventurière à la recherche d'un trésor en explorant un monde remplit d'ennemies et d'énigmes. Un jeu qui me permettra de voir à quel point créer un jeu est un projet ambitieux même si celui-ci est petit.

1.2 Objectifs

Les objectifs de ce projet seront :

- Avoir un système de déplacement avec les touches fléchées
  - Avoir une animation du joueur lors du déplacement

- Avoir un système de point de vie
  - Afficher la barre de point de vie
  - Pouvoir prendre des dégâts diminuant ainsi les points de vie
  - Mourir lorsque les points de vie tombent à zéro
  - Pouvoir se soigner

- Avoir un système d'inventaire
  - Pouvoir ramasser des items et les garder avec soi
  - Pouvoir sélectionner un item spécifique
  - Pouvoir utiliser les items stockés dans l'inventaire
  - Pouvoir équiper un item

- Avoir un système de collision
  - Ne pas pouvoir traverser les murs, les objets et les bords de l'écran
  - Ne pas pouvoir traverser les ennemies avec le système de « hitbox »
  - Gérer les interactions avec les portes qui s'ouvrent

- Avoir un système d'attaque
  - Pouvoir à attaquer au contacte
  - Dégâts variables selon l'arme

- Avoir un système de changement de map

- Avoir un système de sauvegarde
  - Sauvegarder manuellement via le menu
  - Sauvegarder automatiquement à chaque changement de map
  - Charger la dernière sauvegarde au démarrage

- Avoir un menu principal et un menu pause

Planification initiale

Pour ma planification initiale j'ai prévu de commencé par les designs du Joueur qui sera animé et du premier Background afin de pouvoir directement commencer à coder la base c'est-à-dire le Joueur en lui-même ensuite dès que celui-ci sera fait je m'attaquerai à la logique des interactions avec les différents objets ainsi que du menu, des collisions et du changement de map et pour finir je terminerai par la Sauvegarde.

Tâches	Sprint 1 (02.02.2026 - 15.02.2026)	Sprint 2 (16.02.2026 - 01.0	Sprint 3
Designe des enemis du joueur et du monde ainsi que des objects	X		
Déplacement du joueur	X		
Attaque du joueur	X		
Inventaire du joueur	X		
interaction du joueur avec les objets	X		
Gestion des point de vie	X		
Ajout des collisions		X	
Ajouts de la logique des interactions des objets		X	
Changement de map		X	
Sauvegarde du monde			X

Analyse / Conception

Concept

Pour ce projet, plusieurs fonctionnalités ont été discutées. Parmi celles discutées voici celles qui ont été choisies :

1. Mouvement du joueur :
   Le joueur pourra se déplacer sur l'écran en utilisant les touches fléchées du clavier, il ne pourra pas traverser les bords de l'écran

2. Point de vie du joueur
   Le joueur possèdera une barre de vie qui s'affichera en haut à gauche de l'écran. Elle sera limitée à 100hp comme dans la plupart des jeux

3. Système d'interaction avec l'environnement pour le joueur
   Le jeu aura un système d'interaction avec l'environnement pour que le joueur puisse acquérir différents objets ou débloquer de nouvelle porte

4. Système d'attaque pour le joueur
   Le jeu comportera un système d'attaque pour que le joueur puisse se défendre lorsque des ennemies s'approche de lui pour l'attaquer

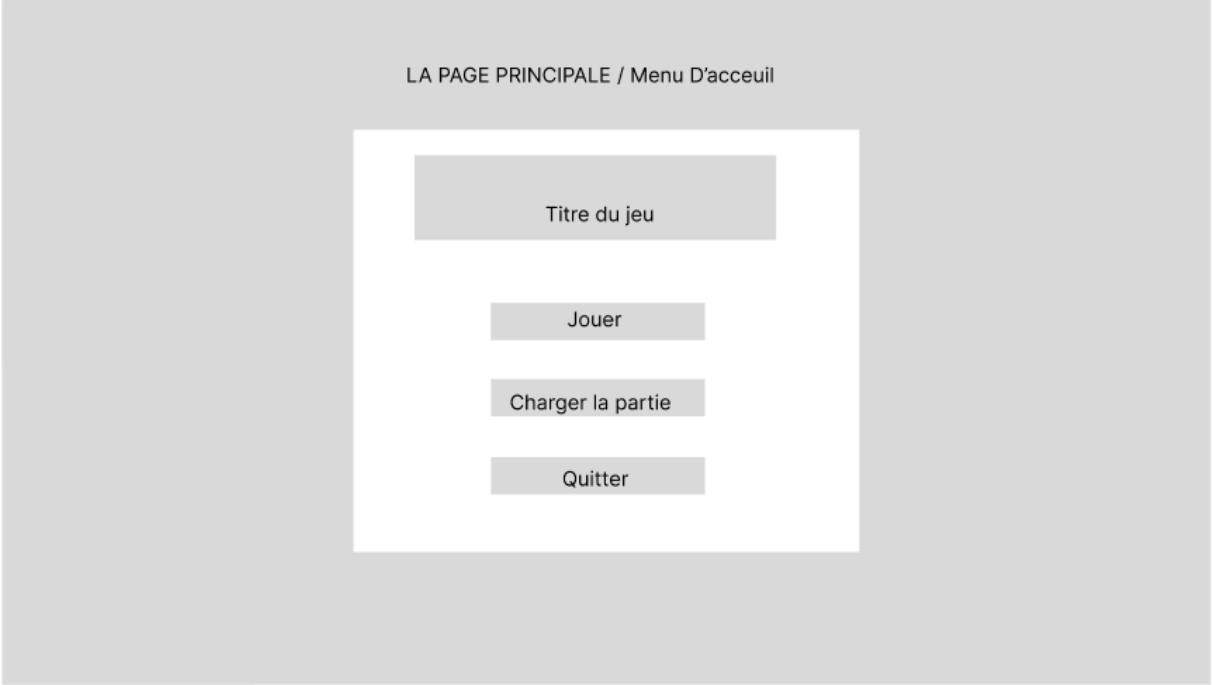
5. Système de soin
   Le jeu comportera un système de soin pour que le joueur puisse se régénérer s'il a pris des dégâts

6. Système de collision
   Le jeu possèdera un système de collision qui empêchera le joueur de traverser les ennemis et les objets limitant ainsi ses déplacements.

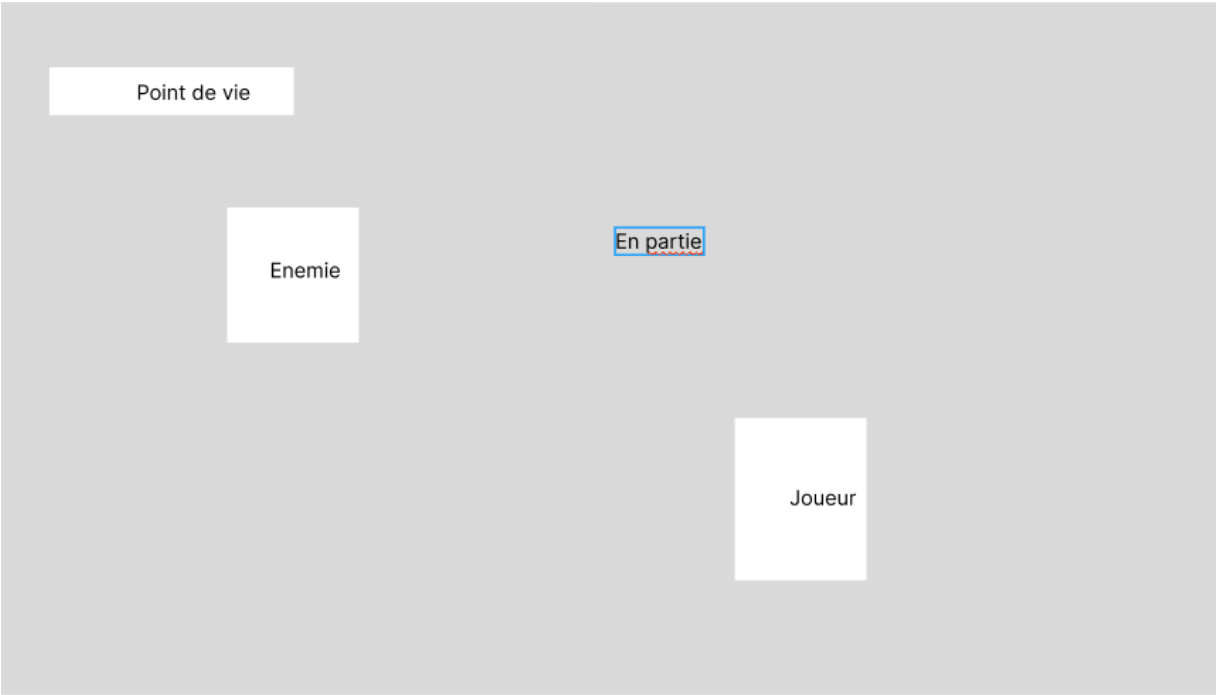
7. Changement de map pour se déplacer dans le monde
   Le monde sera illustré sous forme de tableau et le joueur pourra s'y déplacer en touchant les bords de l'écran

8. Un Système de sauvegarde
   Le jeu possèdera un système de sauvegarde qui sauvegardera l'état du monde la position du joueur ses points de vie son inventaire et son équipement

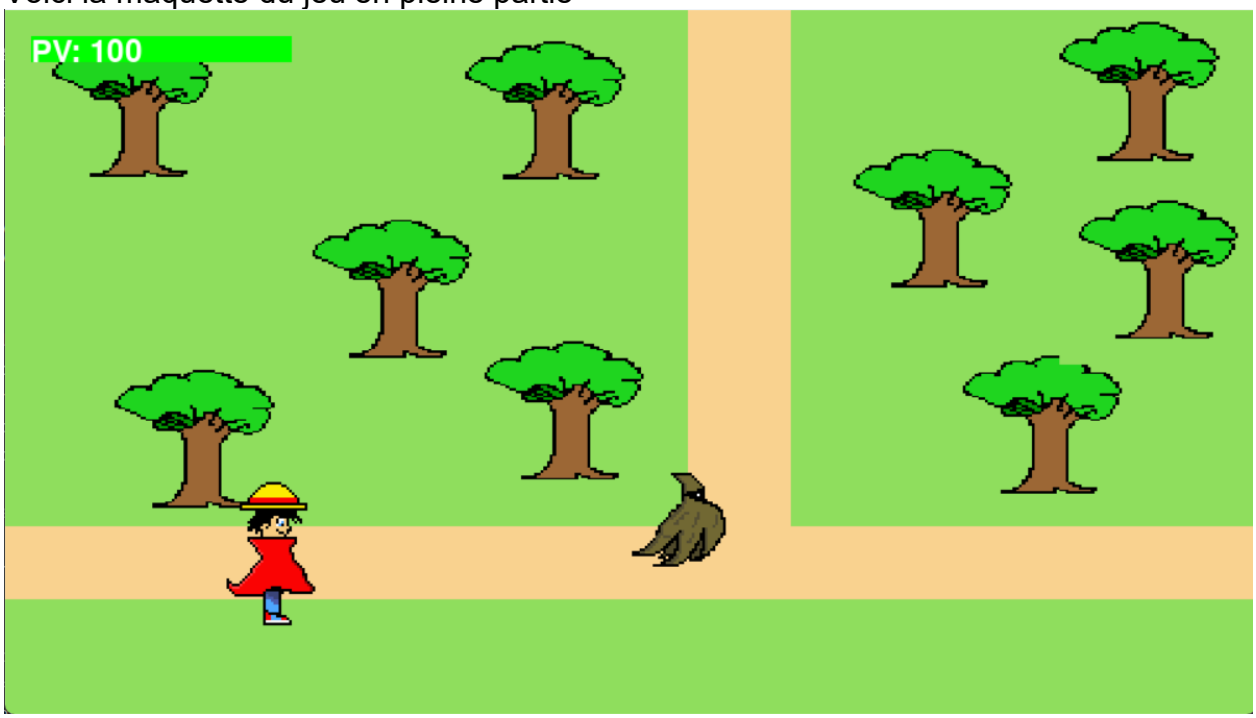
Maquettes :



Voici une maquette qui montre à quoi ressemblera le menu principal lorsqu'on lance le jeu



Voici la maquette du jeu en pleine partie



Voici une représentation plus visuelle du jeu en partie

Diagramme de Classe :

Choix de code :

Collision joueur (hitbox) :

J'ai décidé de gérer les collisions du joueur avec le système de hitbox qui permettra de gérer plus facilement les attaques de l'ennemie aussi pour avoir un rendu visuel plus réaliste. Comme la hitbox est un rectangle qui se forme seulement autour de la partie dessinée de la Sprite lorsque qu'on la configure et non toute la Sprite, on peut l'utiliser pour simuler une collision réaliste avec les ennemies. L'autre avantage de la hitbox est que, comme c'est un rectangle le code sera moins lourd que si on faisait calculer l'ordinateur aux pixels près pour avoir un meilleur résultat.

Déplacement de l'ennemi :

En ce qui concerne les déplacements de l'ennemi, j'utilise la librairie de math pour utiliser la fonction « math.hypo » qui me permettra de calculer la distance du vecteur entre les deux entités peu importe où elle se trouve et ainsi pouvoir choisir à partir de quelle distance je veux qu'il se rapproche de la cible.

Changement de map :

Pour le changement de map, le but sera de créer une fonction qui saura reconnaitre quelle map est en cours pour qu'elle puisse choisir les bonnes suites de map. La fonction dépendra donc des arguments qu'elle aura

Stratégie de test

En ce qui concerne ma stratégie de test, tous mes tests se feront de manière manuelle car mes fonctions ne peuvent fonctionner si le jeu n'est pas lancé. Il y aura donc plusieurs tests unitaires pour vérifier le bon fonctionnement de la fonction. Ensuite à la fin un test d'intégration pour tester qu'il n'y a pas de conflits avec les autres fonctions

Risques techniques

Plusieurs risques peuvent surgir lors de ce projet. Cependant parmi celles-ci, celles qui ont plus de chance de surgir sont :

- Pertes de données si on me vol mon pc ou si tout mon projet est corrompu.
- L'incompatibilité sur MAC OS ou Linux car je code uniquement sur Windows 10
- Le fait que le client ne puisse pas lancer car il se pourrait qu'il n'ait pas installé les dépendances

Planification

Pour le premier sprint je commence par intégrer le design des persos des ennemies et du background afin d'avoir un visuel sur le code et ensuite les fonctionnalités de base du joueur qui sont les déplacements et ses collisions. J'affiche aussi sa barre de vie et je commence à gérer l'ennemie.

Pour le deuxième sprint je commence à agrandir le monde pour intégrer le changement de map, la gestion de l'inventaire, et de l'interaction du joueur avec certains objets ce qui implique la création des objets.

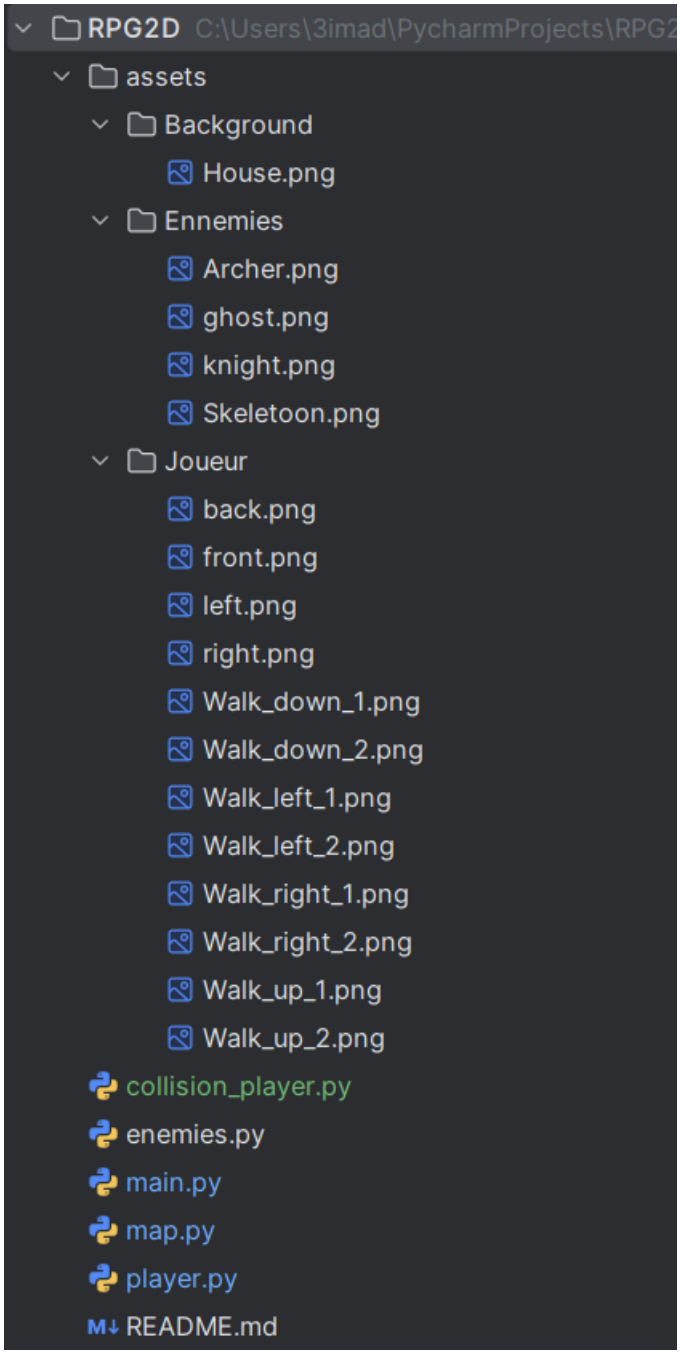
Le troisième sprint regroupera les interactions du joueur avec les items et les ennemies, donc c'est-à-dire, reprendre le système d'inventaire afin qu'il fonction pour tout les type d'objets que l'on ajoutera dans le jeu, concevoir les portes pour que le joueur puisse interagir avec et gérer les armes et les attaques du joueur. Ensuite il restera le menu de départ et de pause et enfin la gestion du système de sauvegarde.

Dossier de conception

OS : Windows 11
Logiciel du design des maps : Pixel Studio version : 5.42.0.0
Logiciel de programmation : JetBrains PyCharm version : 2025.2.1
Langage de programmation : Python version : 3.13.9
Documentation : Word Office version : 2602
Desing de la maquette : Figma version en ligne
Dépôt : Git version 2.51.2
Diagramme de classe : https://lucid.app/lucidchart/decc2c58-4c6f-48be-9ac1-b38fe976cb2d/edit?beaconFlowId=A0E0DDBD1EB3FF73&invitationId=inv_8feadd20-0f56-46a1-935a-df7ab8931388&page=HWEp-vi-RSFO#

Réalisation

Dossier de réalisation



Description des tests effectués plus résultats des tests d'acceptations

Mouvement du joueur (flèche du haut) :

Description :

Lorsque l'on appuie sur la flèche du haut, le personnage se dirige vers le haut

Test d'acceptation :

Player goes up

Given : The game is started

When : Player press the key arrow up

Then : player goes up

- [x] Réussi

Mouvement du joueur (flèche du bas) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers le bas

Test d'acceptation :

Player goes down

Given : The game is started

When : Player press the key arrow down

Then : player goes down

- [x] Réussi

Mouvement du joueur (flèche de gauche) :

Description :

Lorsque l'on appuie sur la flèche gauche, le joueur se dirige vers la gauche.

Test d'acceptation :

Player goes left

Given : The game is started

When : Player press the key left arrow

Then : player goes left

- [x] Réussi

Mouvement du joueur (flèche de droite) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers la droite.

Test d'acceptation :

Player goes right

Given : The game is started

When : Player press the key right arrow

Then : player goes right

- [x] Réussi

Collision entre le joueur et les bords de l'écran :

Description :

Lorsque l'on se dirige vers n'importe quels bords de l'écran non relié vers une autre map, il se retrouve bloqué et ne peut plus continuer dans cette direction.

Test d'acceptation :

Collision with borders

Given :

Player is in the game

When :

He touches the borders of the screen

Then :

He cannot go through

- [x] Réussi

Collision entre le joueur et les objets :

Description :

Lorsque le joueur se heurte avec un objet du décor, il ne peut pas le traverser et dois donc changer de direction.

Test d'acceptation :

Collision with objects:

Given :

Player is in the game

When :

He touches any objects in the game

Then :

He cannot go through it

- [x] Réussi

Ennemi qui suit le joueur lorsqu'il est proche par le haut :

Description :

Lorsque l'ennemi se trouve au-dessus du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre

Test d'acceptation :

enemy goes down

Given :

Player and enemy are in the game

When :

player is under and close to the enemy

Then :

the enemy start chasing him by going down

- [x] Réussi

Ennemi qui suit le joueur par le bas :

Description :

Lorsque l'ennemi se trouve en dessous du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes up

Given :

Player and enemy are in the game

When :

player is above and close to the enemy

Then :

the enemy start chasing him by going up

- [x] Réussi

Ennemi qui suit le joueur par la gauche :

Description :

Lorsque l'ennemi se trouve à gauche du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes left

Given :

Player and enemy are in the game

When :

player is tand close to the enemy

Then :

the enemy start chasing him by going to the left

- [x] Réussi

Ennemi qui suit le joueur lorsqu'il est proche par la droite :

Description :

Lorsque l'ennemi se trouve à droite du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes right

Given :

Player and enemy are in the game

When :

player is the right side and close to the enemy

Then :

the enemy start chasing him by going right

- [x] Réussi

Collision entre l'ennemi et le joueur :

Description :

Lorsque l'ennemi entre en collision avec le joueur, les deux entités ne peuvent se traverser et l'ennemi attaque le joueur toute les 1 secondes en faisant descendre ses points de vie par 10.

Test d'acceptations :

Collision with ennemies

Given :

Player is in the game

When :

He touches a enemy

Then :

He cannot go through it

- [x] Réussi

enemy touches player

Given :

Player and enemy are in the game

When :

enemy touches player

Then :

player start taking damage

- [x] Réussi

Ennemi qui se trouve loin du joueur :

Description :

Lorsque l'ennemi se trouve loin du joueur, celui-ci ne bouge plus de sa place jusqu'à ce que le joueur se trouve à nouveau proche de lui.

Test d'acceptation :

player is far from enemy

Given :

Player and enemy are in the game

When :

player is far from enemy

Then :

the enemy don't chase the player

- [x] Réussi

Changement de map depuis le haut de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le haut, il peut la traverser et se retrouve sur le bas la nouvelle map.

Test d'acceptation :

Link up to down

Given :

Two background connected by sides

When :

Player go to the top side of one background

Then :

He appears in the next background to the bottom

- [x] Réussi

Changement de map depuis le bas de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le bas, il peut la traverser et se retrouve sur le haut la nouvelle map.

Test d'acceptation :

Link down to up

Given :

Two background connected by sides

When :

Player go to the bottom side of one background

Then :

He appears in the next background to the top

- [x] Réussi

Changement de map depuis la gauche de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le haut, il peut la traverser et se retrouve sur le côté droite de la nouvelle map.

Test d'acceptation :

Link left to right

Given :

Two background connected by sides

When :

Player go the very left side of one background

Then :

He appears in the next background to the right

- [x] Réussi

Changement de map depuis la droite de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par la droite, il peut la traverser et se retrouve sur le côté gauche de la nouvelle map.

Test d'acceptation :

Link right to left

Given :

Two background connected by sides

When :

Player go the very right side of one background

Then :

He appears in the next background to the left

- [x] Réussi

Map non connecté par le haut :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis le haut, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the top

Given :

A background is not connected to the top

When :

Player go to the top side of the background

Then :

He still next to the top border and the background doesn't change

- [x] Réussi

Map non connecté par le bas :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis le bas, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the bottom

Given :

A background is not connected to the bottom

When :

Player go to the bottom side of the background

Then :

He still next to the bottom border and the background doesn't change

- [x] Réussi

Map non connecté par la gauche :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis la gauche, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the left

Given :

A background is not connected to the left

When :

Player go to the left side of the background

Then :

He still next to the left border and the background doesn't change

- [x] Réussi

Map non connecté par la droite :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis la gauche, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the right

Given :

A background is not connected to the right

When :

Player go to the right side of the background

Then :

He still next to the right border and the background doesn't change

- [x] Réussi

Affichage de l'inventaire :

Description :

Lorsque le joueur appuie sur la touche « I », l'inventaire s'affiche

Test d'acceptation :

Inventory opens

Given :

Player is in the game

When :

Player press the key "I"

Then:

Inventory opens

- [x] Réussi

Inventaire vide :

Description :

Lorsque l'inventaire est vide et que le joueur l'affiche il y est écrit « Inventaire est vide »

Test d'acceptation :

Empty Inventory

Given :

A player with an empty inventory

When :

Player press the key I

Then :

Inventory opens and says *Inventaire est vide*

- [x] Réussi

Sélection des items dans l'inventaire :

Description :

Le joueur peut sélectionner l'item qu'il souhaite utiliser en les flèches haut et bas du clavier

Test d'acceptation :

Select an Item

Given :

Player is in the Inventory with a lot of Items in it

When :

Player press the keys Up ou Down

Then :

He can choose an Item

- [x] Réussi

Utilisation de l'item puis disparition de celui-ci :

Description :

Lorsque le joueur utilise un item en appuyant sur la touche « E », ce dernier fait effet avant de disparaître de l'inventaire

Test d'acceptation :

Using an Item

Given :

Player is in the Inventory with an Item

When :

Player press the key E

Then :

Player uses the Item and it dissappears

- [x] Réussi

Ramassage des items plus stockage dans l'inventaire :

Description :

Test d'acceptation :

1.1 Collision du joueur avec l'environnement :

Lorsque le joueur se déplace sur la map, celui-ci peut se trouver bloquer s'il percute un mur ou un des objets de la map.

Codes Particuliers

Ennemi qui suit le Joueur :

Pour faire en sorte que l'ennemi suit le joueur peut importe sa position par rapport au joueur, j'ai utilisé la librairie de math déjà intégré dans Python, pour utiliser la fonction hypoténuse, qui me permettra de calculer la distance entre le joueur et l'ennemi en temps réel et ainsi simuler une poursuite.

Code :

```
def move(self, player):  1 usage (1 dynamic)  riffuu1 *
    if not self.actif:
        return  # ← Ne fait rien si l'ennemi est mort

    dx = player.rect.centerx - self.rect.centerx
    dy = player.rect.centery - self.rect.centery

    distance = math.hypot( *coordinates: dx, dy)
    if distance == 0:
        return

    if distance <= 200:  # évite division par zéro
        dx /= distance
        dy /= distance

        future_hitbox = self.hitbox.copy()
        future_hitbox.x += dx * self.vitesse
        future_hitbox.y += dy * self.vitesse

        if not future_hitbox.colliderect(player.hitbox):
            self.rect.x += dx * self.vitesse
            self.rect.y += dy * self.vitesse

            self.hitbox.x = self.rect.x + 40
            self.hitbox.y = self.rect.y + 70
```

Sur cette image vous pouvez voir une partie du code du déplacement de l'ennemi, l'autre partie étant lorsqu'elle attaque l'ennemi. Le code commence avec le calcule

Changement de map :

Lorsque j'ai commencé à tester ma fonction « switch_map » pour changer de map, celle-ci a posé quelques difficultés du fait qu'il fallait être attentif sur la position du joueur ainsi que du background, afin de pouvoir placer la prochaine map voulu. Ces difficultés on, cependant, était résolu assez rapidement pour avoir une personne qui se déplace dans son propre monde.

```
@staticmethod   1 usage   riffuu1 *
def switch_map(current_map,player,map1,map2,map3,House):
    # Sortie à gauche
    if player.rect.x ==0:
        if current_map == map1:
            player.rect.x = map2.width - player.rect.width
            return map2


    # Sortie à droite
    elif player.rect.x == 800:
        if current_map == map2:
            player.rect.x = 0
            return map1

    #Sortie en haut
    elif player.rect.y ==0:
        if current_map == map2:
            player.rect.y = 410
            return map3

    #Sortie en bas
    elif player.rect.y  == 412:
        if current_map == House:
            player.rect.y = 300
            return map1
        if current_map == map3:
            player.rect.y = 0
            return map2
    return current_map
```

Cette image illustre la fonction « switch_map ». À l'intérieur de cette fonction on retrouve 4 blocs et chaque bloc représente un sens de direction (gauche, droite, haut, bas). À l'intérieur de ces blocs se trouve ensuite une condition if qui cherche à savoir le background actuel afin de pouvoir utiliser par la suite la prochaine bonne image.

1.5 Les Collisions :
Gérer les collisions a été un vrai défi car au début j'ai commencé par le faire par pixel ce qui marchaient assez puis afin de pouvoir créer une collision réelle entre le joueur et les ennemies il a fallu revoir tout mon système de collision pour à la fin, opter pour les hitboxs qui ont été beaucoup plus utiles à la fin

Erreurs restantes

Liste des documents fournis

Conclusions

Annexes

Résumé du rapport du TPI / version succincte de la documentation

Sources – Bibliographie

https://www.techwithtim.net/tutorials/game-development-with-python/pygame-tutorial/pygame-enemies

https://www.youtube.com/watch?v=whDf00oeI-E&list=PLkkm3wcQHjT7gn81Wn-e78cAyhwBW3FIc&index=7

https://www.youtube.com/watch?v=MRalNWVSqrc

https://www.youtube.com/watch?v=QLPRsWCQ5BE

Journal de travail

Date	Durée	Activité	Remarques
02.02.2026	4h	Planification du projet	
02.02.2026	30min	Création du joueur	
03.02.2026	1h30	Création du joueur	
03.02.2026	1h30	Début Code Player + plus Premier Background	
03.02.2026	1h30	Documentation	
04.02.2026	1h10	Meeting avec chef de projet	
04.02.2026	20min	Design enemie	
09.02.2026	20min	Afficher la barre de vie	
09.02.2026	20min	Mouvement du personnage avec ses limites sur l'écran	
09.02.2026	20min	Design Objets	
09.02.2026	1h	Class Enemy	
10.02.2026	2h45	Move enemy	
10.02.2026	3h	Collision	
11.02.2026	2h	Documentation	
23.02.2026	1h30	Documentation plus IceScrum	
24.02.2026	1h30	Documentation + désign des différents objets (épée, potion, clé..)	
24.02.2026	2h15	Création de la class objet plus l'inventaire et commencement du menu principal	
02.03.2026	45 min	Documentation	

02.03.2026	30 min	Changement de gestion de projet : IceScrum -> Git Projects	La raison est que IceScrum c'est bloqué une fois et depuis plus de nouvelle
02.03.2026	1h45	Désign de nouvelles maps	
03.03.2026	1h20	Ajout de la fonction du changement de map	
03.03.2026	1h30	Amélioration des déplacements de l'ennemi (Ajout de son attaque au contacte	
04.03.2026	20 min	Meeting avec le chef de projet	
04.03.2026	20 min	Gestion du planning sur Git Project	
09.03.2026	1h15	Gestion de projet	
09.03.2026	30min	Readme	
10.03.2026	1h30	Création de la class et mise à jour de l'inventaire	
10.03.2026	45 min	Affichage de l'inventaire plus test Et ramassage des objets ramassables plus test	
10.03.2026	45 min	Affichage de l'image de l'item dans l'inventaire	
11.03.2026	45min	Documentation	
11.03.2026	45min	Meeting avec chef de projet	
11.03.2026	45min	Documentation	
16.03.2026	3h	Documentation	
17.03.2026	45min	Documentation	
17.03.2026	45min	Gestion de l'arme	
17.03.2026	2h15	Reparation de l'inventaire	
18.03.2026	45min	Meeting avec Chef de Projet	
18.03.2026	45 min	Documentation Codes Particuliers	

Manuel d'Installation

Manuel d'Utilisation

Archives du projet